

Feature Extraction from ECG Signal and Detection of Ventricular Fibrillation

EEE 312
DSP PROJECT
REPORT

GROUP: 01

2006133-
SYMRIA RAIHAN
2006134-
S.M RAZIBUR RAHMAN
2006136-
BASHIR SHAHRIYER
2006153-
RIAJUL KARIM
CHOWDHURY

BANGLADESH UNIVERSITY OF ENGINEERING AND TECHNOLOGY
DEPARTMENT OF ELECTRICAL AND ELECTRONIC ENGINEERING

EEE 312 (July 2023)

Digital Signal Processing Laboratory 1

Final Project Report

Section: C1 Group: 01

Feature Extraction from ECG Signal and Detection of Ventricular Fibrillation

Course Instructors:

Shahed Ahmed
Lecturer, Department of EEE

Md. Obaidur Rahman
Lecturer, Department of EEE

Academic Honesty Statement:

IMPORTANT! Please carefully read and sign the Academic Honesty Statement, below. Type the student ID and name, and put your signature. *You will not receive credit for this project experiment unless this statement is signed in the presence of your lab instructor.*

"In signing this statement, We hereby certify that the work on this project is our own and that we have not copied the work of any other students (past or present), and cited all relevant sources while completing this project. We understand that if we fail to honor this agreement, We will each receive a score of ZERO for this project and be subject to failure of this course."

Full Name: Md. Symria Raihan **Student ID: 2006133**	**Full Name: S. M. Razibur Rahman** **Student ID: 2006134**
Full Name: Bashir Shahriyer Rafi **Student ID: 2006136**	**Full Name: Riajul Karim Chowdhury** **Student ID: 2006153**

Table of Contents

1 Abstract 1

2 Introduction 1

3 Design 2

3.2 Algorithm Overview 4

3.2.1 QRS detection: Pan-Tompkins Algorithm 4

3.2.2 Ventricular Fibrillation Detection: Threshold Crossing Sample Count Algorithm 5

4 Implementation 7

4.1 Implementation: Output 16

5 Design Analysis and Evaluation 22

5.1 Limitations of Tools (PO(e)) 22

6 Reflection on Individual and Teamwork (PO(i)) 23

7 Future Work (PO(l)) 23

8 References 25

Figure 1: Block Diagram of Pan-Tompkins Algorithm 4
Figure 2 Original ECG signal 5
Figure 3 BPF output 5
Figure 4 Derivative Block output 5
Figure 5 Integration output 5
Figure 6 TCSC algorithm 5
Figure 7 Healthy Person ECG 16
Figure 8 Processing Output 17
Figure 9 ECG with Q,R,S location 22

1 Abstract

Our project focuses on the implementation of the Pan-Tompkins algorithm for QRS detection and the identification of Ventricular Fibrillation (VF) in electrocardiogram (ECG) signals. The problem involves processing noisy biomedical data to extract clinically relevant information. The “Pan-Tompkins Algorithm's” offers a simple yet effective solution, though it requires careful parameter tuning. Alternative methods include threshold-based algorithms, template matching, and machine learning approaches. Our project aims to evaluate the “Pan-Tompkins Algorithm's” performance using MATLAB, demonstrating its accuracy in QRS detection .For Ventricular Fibrillation (VF) detection, we used “Threshold Crossing Sample Count Algorithm” which is a simple time domain approach offering high levels of accuracy. We also explore potential enhancements to address real-world challenges in cardiac monitoring and diagnosis.

2 Introduction

The accurate detection and analysis of cardiac signals play a pivotal role in the diagnosis and management of cardiovascular diseases. Among various cardiac signals, the electrocardiogram (ECG) is widely utilized for its ability to provide valuable insights into the electrical activity of the heart. Specifically, the identification of QRS complexes, representing ventricular depolarization, and the detection of Ventricular Fibrillation (VF), a life-threatening cardiac arrhythmia, are essential tasks in ECG signal processing.

Our project focuses on implementing the Pan-Tompkins algorithm for QRS detection and VF identification in ECG signals. The primary objective is to develop a robust and efficient solution for automated cardiac signal analysis. This project addresses the challenge of accurately extracting QRS features and detecting VF episodes from noisy and complex ECG recordings.

The problem at hand is inherently complex due to several factors. Firstly, ECG signals are susceptible to various artifacts and noise sources, including baseline wander, muscle noise, and electrode motion artifacts. These noise sources pose significant challenges to the accurate detection of QRS complexes and the identification of abnormal cardiac rhythms. Secondly, the detection of VF requires high sensitivity and specificity to ensure timely intervention and patient survival. VF episodes manifest as irregular and chaotic patterns in the ECG, further complicating the detection task.

Several alternative solutions exist for QRS detection and VF identification in ECG signals. Traditional methods include threshold-based algorithms, which rely on predefined amplitude thresholds to detect QRS complexes. Template matching techniques match predefined templates to segments of the ECG signal to identify QRS complexes. Machine learning approaches, such as support vector machines and artificial neural networks, have also been employed for automated QRS detection and VF identification.

While these alternative methods offer different advantages and trade-offs in terms of

computational complexity, accuracy, and robustness, the Pan-Tompkins algorithm is chosen for its simplicity and widespread adoption in clinical practice. The Pan-Tompkins algorithm provides a real-time solution for QRS detection by incorporating signal processing techniques such as filtering, differentiation, and thresholding.

Our project aims to:
- Implement the Pan-Tompkins algorithm for QRS detection in MATLAB.
- Evaluate the performance of the Pan-Tompkins algorithm using publicly available ECG datasets.
- Implement "Threshold Crossing Sample Count Algorithm" to detect VF episodes and evaluate its performance in VF detection.

By addressing these objectives, our project seeks to contribute to the advancement of automated cardiac signal analysis techniques and facilitate early detection and intervention in cardiac emergencies.

3 Design

3.1.1 Identification of Scope

The scope of this project encompasses the extraction of relevant features from electrocardiogram (ECG) signals for analyzing cardiac health. Specifically, the project focuses on the detection and analysis of various parameters including R peaks, QRS intervals, and heart rate. The project utilizes the classic Pan-Tompkins algorithm for this. Additionally, the project aims to implement a disease detection algorithm known as the Threshold Crossing Sample Count (TCSC) algorithm for identifying ventricular fibrillation.
The project involves signal processing techniques applied to ECG data obtained from online databases. By utilizing digital signal processing (DSP) methodologies, the project seeks to accurately detect and analyze critical features within the ECG signal. The extracted features play a crucial role in diagnosing cardiac abnormalities and monitoring overall cardiac health. Through the implementation of the TCSC algorithm, the project aims to enhance the capability of automated disease detection, particularly for ventricular fibrillation, a life-threatening cardiac arrhythmia. By establishing thresholds and analyzing sample counts, the algorithm offers a robust method for identifying abnormal ECG patterns associated with ventricular fibrillation.
Overall, the scope of this project encompasses the application of DSP techniques for feature extraction, analysis, and disease detection in ECG signals, with a specific focus on ventricular fibrillation using the TCSC algorithm.

3.1.2 Literature Review

Recent research in the analysis of electrocardiogram (ECG) signals and the detection of cardiac abnormalities has seen significant advancements, driven by the integration of digital signal processing (DSP) techniques, machine learning algorithms, and the availability of large-scale datasets. This section provides an overview of recent studies and developments in this field.
R peak and other features have been detected classically by the well-known Pan-Tompkins algorithm. The very popular algorithm has been modified in several ways to meet the demand of any particular case. Although the algorithm was proposed in the early 1990s, it's still in use due to its effectiveness. Rapid advancements in signal processing and machine learning have

led to the emergence of novel techniques for R peak detection in ECG signals. Recent studies have explored deep learning architectures, such as convolutional neural networks (CNNs) and recurrent neural networks (RNNs), for robust and accurate R peak detection in noisy and low-quality ECG signals. These deep learning-based approaches leverage hierarchical feature representations and sequential modeling to improve detection performance, demonstrating superior results compared to traditional methods.

In addition to R peak detection, recent research has focused on enhancing the accuracy and efficiency of QRS complex detection algorithms. Advanced signal processing techniques, such as adaptive filtering and wavelet transforms, have been combined with machine learning algorithms to develop adaptive and data-driven approaches for QRS complex detection. These methods adaptively adjust to variations in signal morphology and noise levels, leading to improved detection performance across diverse ECG datasets.

Heart rate estimation from ECG signals remains a critical task in cardiac monitoring and diagnosis. Recent studies have investigated novel approaches for heart rate estimation, including non-linear dynamics analysis, time-frequency analysis, and deep learning-based regression models. These approaches leverage the temporal and spectral characteristics of ECG signals to accurately estimate heart rate, even in the presence of arrhythmias and noise artifacts.

The detection of ventricular fibrillation, a life-threatening cardiac arrhythmia, continues to be a focus of research in ECG signal analysis. Recent studies have proposed innovative algorithms for ventricular fibrillation detection, incorporating advanced signal processing techniques and machine learning methodologies. Deep learning models, including CNNs and long short-term memory (LSTM) networks, have shown promise in automatically detecting ventricular fibrillation patterns from ECG signals with high sensitivity and specificity.

Furthermore, the development of large-scale ECG databases, such as the PhysioNet repository, has facilitated collaborative research and benchmarking of algorithms for ECG analysis and disease detection. These datasets provide researchers with access to diverse patient populations and a wide range of cardiac conditions, enabling the development and validation of robust algorithms for clinical applications.

Overall, recent research in ECG signal analysis has witnessed significant advancements, driven by innovations in signal processing, machine learning, and data availability. These developments hold great promise for improving the accuracy, efficiency, and clinical utility of ECG-based cardiac monitoring and diagnosis in various healthcare settings.

3.1.3 Formulation of Problem

The formulation of the problem in this project revolves around the accurate extraction of relevant features from electrocardiogram (ECG) signals and the subsequent detection of cardiac abnormalities, particularly ventricular fibrillation, using the Threshold Crossing Sample Count (TCSC) algorithm.

One of the primary challenges is to develop robust algorithms for detecting key features in ECG signals, such as R peaks and QRS complexes, amidst noise and variability in signal morphology. This requires the design and implementation of signal processing techniques capable of effectively filtering noise and enhancing the signal-to-noise ratio, thus enabling accurate feature detection.

Another aspect of the problem formulation is the development and optimization of the TCSC algorithm for ventricular fibrillation detection. This involves setting appropriate thresholds and determining the sample count criteria for identifying abnormal ECG patterns indicative of ventricular fibrillation. Additionally, the algorithm must be evaluated and validated using diverse datasets to ensure its reliability and generalizability in clinical settings.

3.1.4 Analysis

The analysis in this project encompasses several key aspects, including the evaluation of

feature extraction techniques, the performance assessment of the TCSC algorithm for ventricular fibrillation detection, and the comparison of different methodologies for ECG signal analysis.

Firstly, the performance of various feature extraction methods, such as peak detection algorithms for identifying R peaks and QRS complexes, will be assessed. This analysis will involve comparing the accuracy, sensitivity, and specificity of different algorithms using benchmark datasets and ground truth annotations. Additionally, the robustness of these algorithms to noise, artifacts, and signal variations will be evaluated to determine their suitability for clinical applications.

Secondly, the TCSC algorithm will be analyzed in terms of its ability to accurately detect ventricular fibrillation from ECG signals. This analysis will involve testing the algorithm on annotated datasets containing both normal and abnormal ECG recordings and evaluating its performance metrics such as sensitivity, specificity, positive predictive value, and negative predictive value. The algorithm will also be compared against existing methods for ventricular fibrillation detection to assess its efficacy and reliability.

Furthermore, the computational efficiency and real-time performance of the proposed algorithms will be analyzed to determine their feasibility for deployment in clinical settings. This analysis will involve measuring the processing time and resource requirements of the algorithms on different hardware platforms and assessing their scalability to handle large volumes of ECG data.

3.2 Algorithm Overview

3.2.1 QRS detection: Pan-Tompkins Algorithm

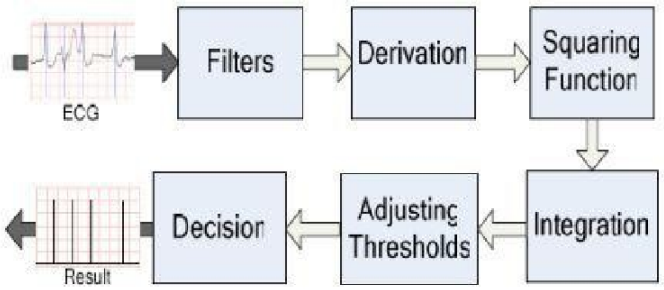


Figure 1: Block Diagram of Pan-Tompkins Algorithm

First, in order to attenuate noise, the signal passes through a digital band pass filter composed of cascaded high-pass and low pass filters. This filter chops off all frequencies above 15Hz and below 5Hz. Fig-2(a) below shows the output of this filter.

The next process after filtering is differentiation followed by squaring and then moving window integration Information about the slope of the QRS is obtained in the derivative stage. The squaring process intensifies the slope of the frequency response curve of the derivative and helps restrict false positives caused by T waves with higher than usual spectral energies. The moving window integrator produces a signal that includes information about both the slope and the width of the QRS complex. Fig. 1(f) is the same as the original ECG in Fig. 2(g)) but the Q,R,S peaks are annotated. Fig-2(e) shows the final output stream of pulses marking the locations of the QRS complexes after application of the thresholds.

The algorithm is divided into three processes: learning phase 1, learning phase 2, and detection. Learning phase 1 requires about 2 s to initialize detection thresholds based upon

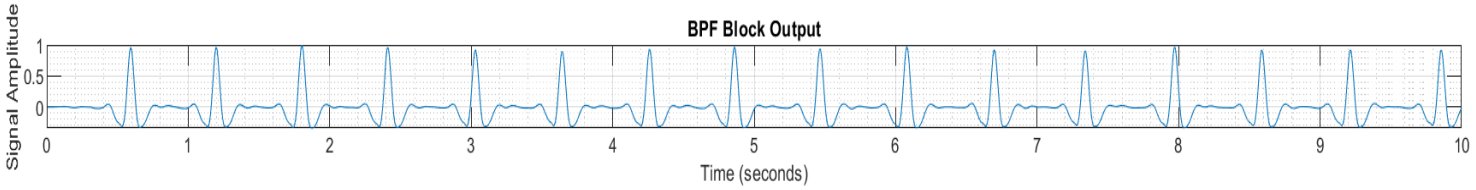


Figure 3 BPF output

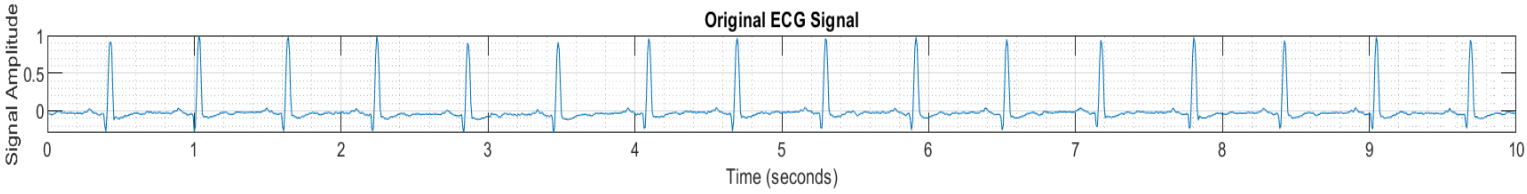


Figure 2 Original ECG signal

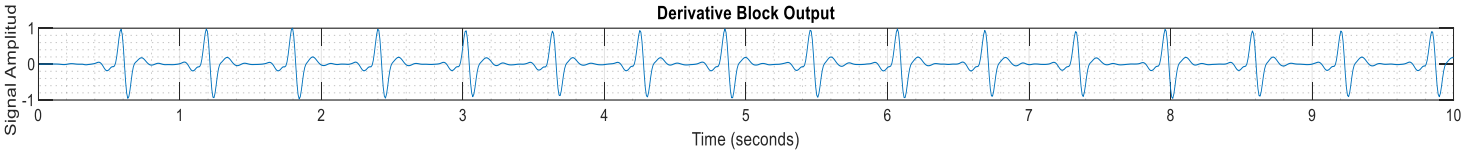


Figure 4 Derivative Block output

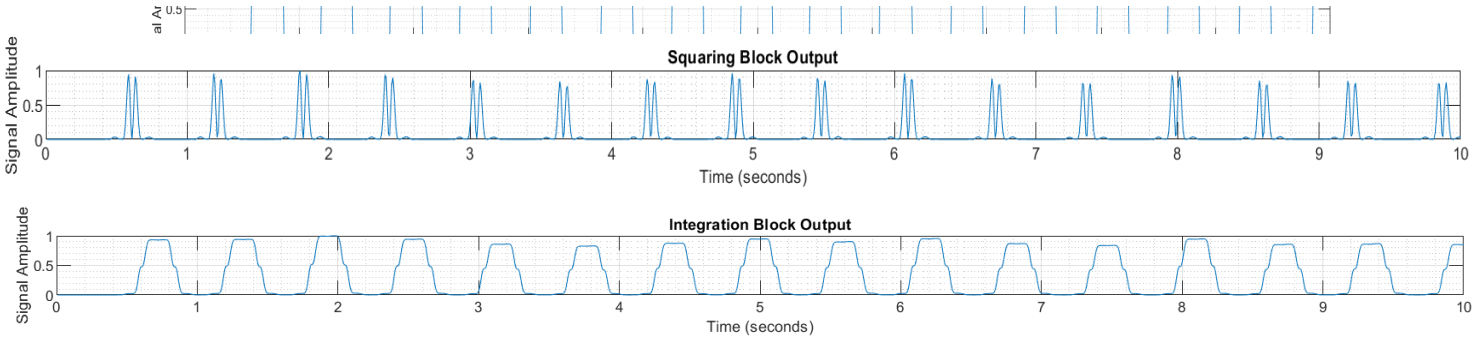
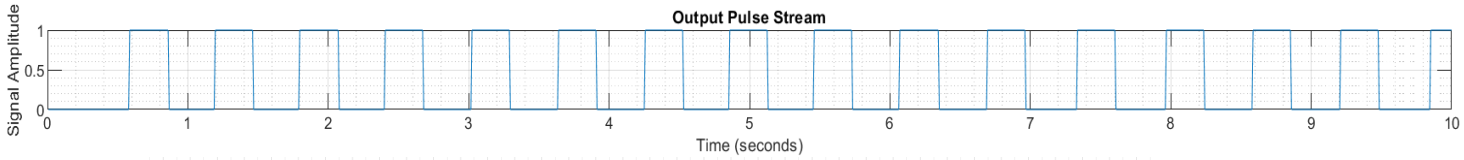


Figure 5 Integration output



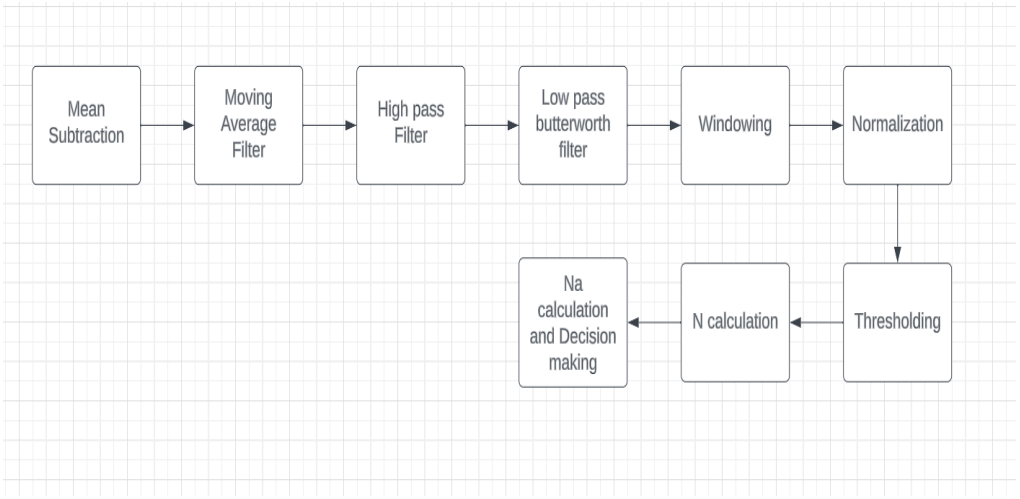


Figure 6 TCSC algorithm

The TCSC algorithm is based on the percent of time the ECG signal remains outside a certain threshold rather than counting pulses. The time is calculated by counting the number of normalized ECG samples that cross the predefined threshold V0. The value of V0 is also suitably selected to be 0.2 after a number of empirical studies. The baseline of the preprocessed ECG signal is assumed to go below this threshold. This assumption is valid for VF and also works well for SR. Since the algorithm uses amplitude normalization, we need to analyze the ECG signal in a shorter window length. In this algorithm, each 3-s ECG segment is investigated and the number of samples that cross the threshold is counted
The TCSC algorithm is described in the following steps:
The ECG signal is preprocessed using the same well known filtering process which is carried out in a MATLAB routine, called filtering.m. The filtering algorithm works in four successive steps.
• First, the mean value is subtracted from the signal.
• Second, a moving average filter of order 5 is applied in order to remove high frequency noise like interspersions and muscle noise.
• Third, a drift suppression is carried out by a high pass filter with a cut-off frequency of 1 Hz.
• In the last step, a low-pass Butterworth filter with a cut-off frequency of 30 Hz is applied which suppresses the high frequency information even more.
Then, each ECG segment is multiplied by a cosine window w(t) which is defined as follows:

$$w(t) = \begin{cases} \frac{1}{2}(1-\cos(4\pi t)) & 0 \le t \le \frac{1}{4} \\ 1 & \frac{1}{4} \le t \le L_s - \frac{1}{4} \\ \frac{1}{2}(1-\cos(4\pi t)) & L_s - \frac{1}{4} \le t \le L_s \end{cases}$$

where Ls(= 3) is the length of the 3-s ECG segment.
The windowed ECG signal is normalized by the absolute maximum value in the segment.
Now, the normalized ECG signal is converted into a binary string b1,b2,b3 ... bn (n being the number of data points) of 0–1 by comparing each data sample xi (i = 1, 2,..., n) with the threshold V0. If the absolute value of xi > $v_{0,}$ then bi = 1, otherwise bi = 0.
Next, N, the percent of data samples that cross V0, is obtained by counting the number of 1s in the binary sequence b = {b1b2b3 ... bn}. N is expressed in percent by the following equation:

$$N = \frac{No.\ of\ Samples\ that\ Cross\ Vo}{Total\ Number\ of\ Samples} * 100$$

Decision is made on every Le-second ECG episode (Le > 3) by averaging Le − 2 consecutive values of N obtained from Le − 2 consecutive 3-s data segments with 1-s step.
The average value Na for an Le-second episode is calculated by the following equation:

$$N_a = \frac{1}{L_e - 2} \sum_{i=1}^{L_e-2} N_i$$

Where Ni is the value of N in the i-th 3-s stage.
If Na is greater than a certain threshold Nd , VF is detected.

4 Implementation

We have divided our work into several functions. The functions are named and their works are given below-

	Function Name	Works
1.	**'bandpass_filtering'**	→ Bandpass filtration
2.	**'differentiator'**	→Differentiating the filtered signal
3.	**'Squaring'**	→Squared the differentiated signal
4.	**'integration'**	→Moving-Window Integration of Squared signal
5.	**'MA'**	→To remove noise by passing the signal by the moving average system
6.	**'thresholding1'**	→Thresholding and detecting the Q peak, R peak, and S peak and their location
7.	**'qrs_dur'**	→To calculate QRS complex duration
8.	**'heart_rate'**	→To calculate heart rate
9.	**'VF_detector'**	→ Detect VF
10.	**'Pan_Tompkins_Implemented_Rpeak_QRSduration'**	→Our main code is written here

The codes of the above functions are given below

1. **'bandpass_filtering':**

```
function[ecg_out2]= bandpass_filtering(ecg)
%% Low pass filtering %%

b=[1 0 0 0 0 0 -2 0 0 0 0 0 1];
a=[1 -2 1] * 32;

%% creating digital filter (Direct form II)
hd1=dfilt.df2(b,a);

%% filtering ECG signal using LPF %%

ecg_out1=filter(hd1,ecg);
ecg_out1=ecg_out1 - mean(ecg_out1);
ecg_out1=ecg_out1/max(abs(ecg_out1));

%% High pass filtering %%

b2=[-1,zeros(1,15),32,-32,zeros(1,14),1];
a2=[1,-1] * 32;

%% creating digital filter (Direct form II)
 hd2=dfilt.df2(b2,a2);

%% filtering the ECG signal from HPF
```

```
ecg_out2 =filter(hd2,ecg_out1);
 ecg_out2= [ecg_out2(1:40)*0.25  ecg_out2(41:end)];
ecg_out2 = ecg_out2/max(abs(ecg_out2));
end
```

2. **'differentiator':**

```
function [ecg_out3]=differentiator(ecg_out2)
%% derivative operator %%

b3= [2 1 0 -1 -2];
a3 = [1] *8;

%% creating digital filter (Direct form II)

hd3 = dfilt.df2(b3,a3);

%% filtering the ECG signal from derivative operator %%

ecg_out3 = filter(hd3,ecg_out2);
%% Normalization %%
ecg_out3=ecg_out3 - mean(ecg_out3);
ecg_out3 = ecg_out3/max(abs(ecg_out3));
end
```

3. **'Squaring':**

```
function [ecg_out4]=squaring(ecg_out3)
%% squaring operation %%

ecg_out4 = ecg_out3.^2;
ecg_out4 = ecg_out4/max(abs(ecg_out4));
end
```

4. **'integration**':

```
function [ecg5]=integration(ecg_out4)
%% moving window integration operation %%

ecg_out4pad = [zeros(1,29) ecg_out4 zeros(1,29)];

for i=30:length(ecg_out4pad)-29
    ecg_int(i-29) = sum(ecg_out4pad(i-29:i))/30;
end

ecg5 = ecg_int';
```

```
ecg5 =ecg5/max(abs(ecg5));
end
```

5. **'MA' :**

```
function filteredsignal =MA(x,M)

z=zeros(1,length(x));

for i=1:M-1
    z(i)=M*x(i);
end

for i= M:length(x)

    for j=0:M-1
    z(i)=z(i)+x(i-j);
    end

end
 filteredsignal =z/M;

end
```

6. **'thresholding1' :**

```
function[Q_val,Q_loc,R_val,R_loc,S_val,S_loc,TH,w]=thresholdin
g1(ecg,ecg5)
% %% Thresholding operation %%
TH = mean(ecg5)*max(ecg5); % Set threshold
w=(ecg5>(TH));
figure
plot(w)
x=find(diff([0 w']) == 1); % Finding location of 0 to 1
transition
y=find(diff([w' 0]) == -1); % Finding location of 1 to 0
transition

%% cancelling the delay due to LOW PASS FILTER and HIGH PASS
FILTER %%
```

```
x=x-(6+16); % 6 DELAY BY LPF & 16 DELAY BY HPF
y=y-(6+16);

%% Detect Q,R,S points %%
for i=1:length(x)
    %% R Locations %%
    [R_val(i),R_loc(i)]=max(ecg(x(i):y(i)));
    R_loc(i) = R_loc(i)-1 + x(i); % adding offset
end
j=1;
for i=1:length(R_loc)
    m=R_val(i);
    n=ecg(R_loc(i)+j);
    while 1
        j=j+1;
        if n>=m
            S_val(i)=m;              %[last e jno jamela na
hoi]
            S_loc(i)=j+R_loc(i)-2;
            break
        elseif j+R_loc(i)>length(ecg)
              break
        else
            m=n;
            n=ecg(j+R_loc(i));
        end
    end
j=1;
end

% DETECTION OF Q PEAK AND CORRESPONDING LOCATIONS
j=1;
for i=1:length(R_loc)
    m=R_val(i);
    n=ecg(R_loc(i)-j);
    while 1
        j=j+1;
        if n>=m
            Q_val(i)=m;
            Q_loc(i)=-j+R_loc(i)+2;
            break
        else
            m=n;
            n=ecg(-j+R_loc(i));
        end
    end
j=1;
end
if length(Q_loc)~=length(S_loc)
    Q_loc(length(Q_loc))=0;
```

```
    S_loc(length(Q_loc))=0;
End
```

7. **'qrs_dur'**:

```
function QRS_duration=qrs_dur(ecg5,R_loc,fs)
flag=0;
filtered_signal = MA(ecg5', 5);
x=diff(filtered_signal);
filtered_signal(find(abs(x)<10^(-1.5)))=0;
for i=1:length(filtered_signal)
 if filtered_signal(i)~=0
     flag=flag+1;
 end
end
QRS_duration=(flag/(2*length(R_loc)))/fs;
End
```

8. **'heart_rate'**:

```
function HT=heart_rate(R_loc,t)
%% Calculation of HEART RATE %%
HT=ceil((length(R_loc)*60)/t(end)); % calculate Heart rate
End
```

9. **'VF_detector'**

```
function Na=VF_detector(ecgsig)

fs = length(ecgsig) / 10;
t = (0:length(ecgsig)-1) / fs;

%% Step 1: Mean subtraction
ecgsig = ecgsig - mean(ecgsig);

% Step 2: Moving average filter
ma_order = 5;
ecgsig_ma = Moving_average(ecgsig, ma_order);

% Step 3: High-pass filter (cut-off frequency = 1 Hz)
hp_filter = designfilt('highpassfir', 'FilterOrder', 50,
'CutoffFrequency', 1, 'SampleRate', fs);
```

```
ecgsig_hp = filter(hp_filter, ecgsig_ma);

% Step 4: Low-pass Butterworth filter (cut-off frequency = 30 Hz)
lp_cutoff = 30;
[b, a] = butter(4, lp_cutoff/(fs/2));
ecgsig_lp = filtfilt(b, a, ecgsig_hp);

% Step 5: Applying cosine window
Ls = 3; % Length of segment (3 seconds)
ns = Ls * fs; % Number of samples in a segment
ne = 1 * fs; % Step size

% Generate cosine window using piecewise function
window = zeros(1, ns);
idx_t = linspace(0, Ls, ns);
idx_1 = idx_t <= (1/4);
idx_2 = idx_t > 1/4 & idx_t < Ls - 1/4;
idx_3 = idx_t >= Ls - 1/4 & idx_t <= Ls;
window(idx_1) = 0.5 * (1 - cos(4 * pi * idx_t(idx_1)));
window(idx_2) = 1;
window(idx_3) = 0.5 * (1 - cos(4 * pi * idx_t(idx_3)));

% Apply cosine window to ECG segments and normalize
ecgsig_normalized = zeros(1, length(ecgsig_lp));
for i = 1:8
    idx_start = (i - 1) * ne + 1;
    idx_end = idx_start + ns - 1;

    ecg_segment = ecgsig_lp(idx_start:idx_end);
    ecg_segment = ecg_segment .* window; % Apply window here
    ecgsig_normalized(idx_start:idx_end) = ecg_segment /
max(abs(ecg_segment));
end

% Step 6: Thresholding and calculating N
V0 = 0.2; % Example threshold
binary_signals = ecgsig_normalized > V0;

N_values = zeros(1, 8);
for i = 1:8
    idx_start = (i - 1) * ne + 1;
    idx_end = idx_start + ns - 1;

    binary_signal = binary_signals(idx_start:idx_end);
    p = sum(binary_signal) / length(binary_signal) * 100;
    N_values(i) = p;
end
Na = mean(N_values);

% Check if Na lies between 25 to 35
```

```
if Na >= 25 && Na <= 35
    disp('VF detected');
elseif Na>=48
    disp('VF detected');
else
    disp('VF not detected');
end
% Plotting
figure;

subplot(3, 1, 1);
plot(t, ecgsig);
title('Original ECG Signal');
xlabel('Time (s)');
ylabel('Amplitude');

subplot(3, 1, 2);
plot(t, ecgsig_normalized); % Adjusted plotting here
title('Normalized ECG Signal');
xlabel('Time (s)');
ylabel('Amplitude');

subplot(3, 1, 3);
stairs(t(1:length(binary_signals)), binary_signals);
title('Binary Signal');
xlabel('Time (s)');
ylabel('Binary Value');
```

10. 'Pan_Tompkins_Implemented_Rpeak_QRSduration' :

```
clc;
clear;
close all;

% Load an ECG signal from a file selected by the user
[filename, pathname] = uigetfile('.', 'Select the
signal');
filewithpath = strcat(pathname, filename);
ecg1 = load(filewithpath);
ecg = ecg1.val;
ecg=ecg-mean(ecg);
ecg=ecg/max(abs(ecg));
fs=length(ecg)/10;
```

```
slen=length(ecg);
t=(1:slen)/fs;
P1=subplot(6,1,1);
plot(t,ecg);
xlim([0,10]);
grid on;
grid minor;
xlabel('Time (seconds)')
ylabel('Signal Amplitude');
title('Original ECG Signal');

%% Bandpass filtering %%
ecg_out2=bandpass_filtering(ecg);
P2=subplot(6,1,2);
plot(t,ecg_out2);
xlim([0,10]);
grid on;
grid minor;
xlabel('Time (seconds)')
ylabel('Signal Amplitude');
title('BPF Block Output');
%% Derivative Operation %%
ecg_out3=differentiator(ecg_out2);
P3=subplot(6,1,3);
plot(t,ecg_out3);
xlim([0,10]);
grid on;
grid minor;
xlabel('Time (seconds)')
ylabel('Signal Amplitude');
title('Derivative Block Output');
%% squaring operation %%
ecg_out4=squaring(ecg_out3);
P4=subplot(6,1,4);
plot(t,ecg_out4);
xlim([0,10]);
grid on;
grid minor;
xlabel('Time (seconds)')
ylabel('Signal Amplitude');
```

```
title('Squaring Block Output');

%% Integration operation %%
ecg5=integration(ecg_out4);
P5=subplot(6,1,5);
plot(t,ecg5);
xlim([0,10]);
grid on;
grid minor;
hold on;
xlabel('Time (seconds)')
ylabel('Signal Amplitude');
title('Integration Block Output');

%% Thresholding operation %%
[Q_val,Q_loc,R_val,R_loc,S_val,S_loc,TH,w]=thresholdin
g1(ecg_out2,ecg5);
% P6=subplot(6,1,6);
figure

plot(t,ecg_out2);
xlim([0,10]);
grid on;
grid minor;
hold on;
plot(t(Q_loc),Q_val,'bo');
plot(t(R_loc),R_val,'r*');
plot(t(S_loc),S_val,'go');
xlabel('Time (seconds)')
ylabel('Signal Amplitude');
title('ECG Signal with QRS Locations after
Thresholding');
legend('ECG','Q','R','S');
hold off;


%% Calculation of Heart Rate and QRS Duration %%
HT=heart_rate(R_loc,t);
disp(['Heart Rate for ecg = ', num2str(HT), '
beats/min']);
```

```
fprintf('\n')
% set(handles.edit1,'string',dur); % Display QRS
Duration
% disp(['QRS Duration for ecg = ' num2str(dur) '
sec']);
QRS_duration=qrs_dur(ecg5,R_loc,fs);

disp([' QRS duration = ' num2str(QRS_duration)
'seconds'])

%% VF detection
fprintf('\n')
Na=VF_detector(ecg)
```

4.1 Implementation: Output

Intput for a Healthy person ECG:

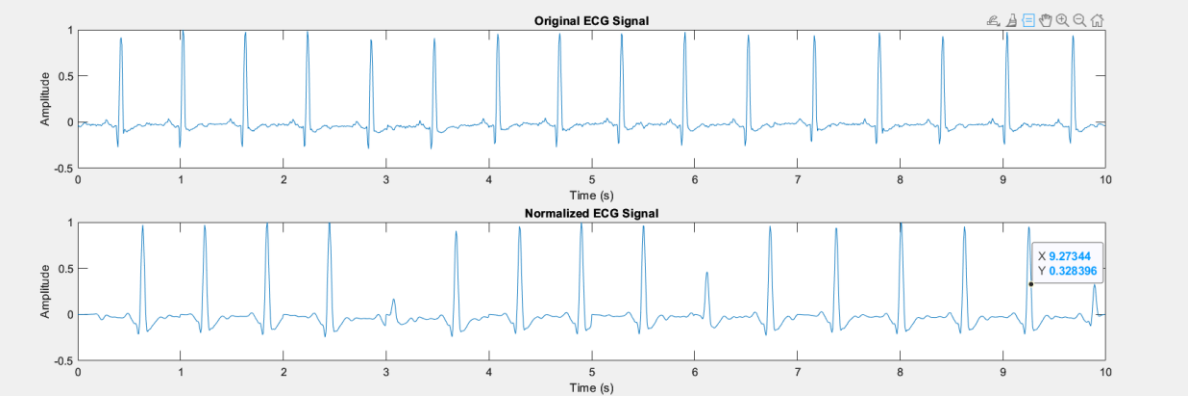


Figure 7 Healthy Person ECG

Preprocessing Output:

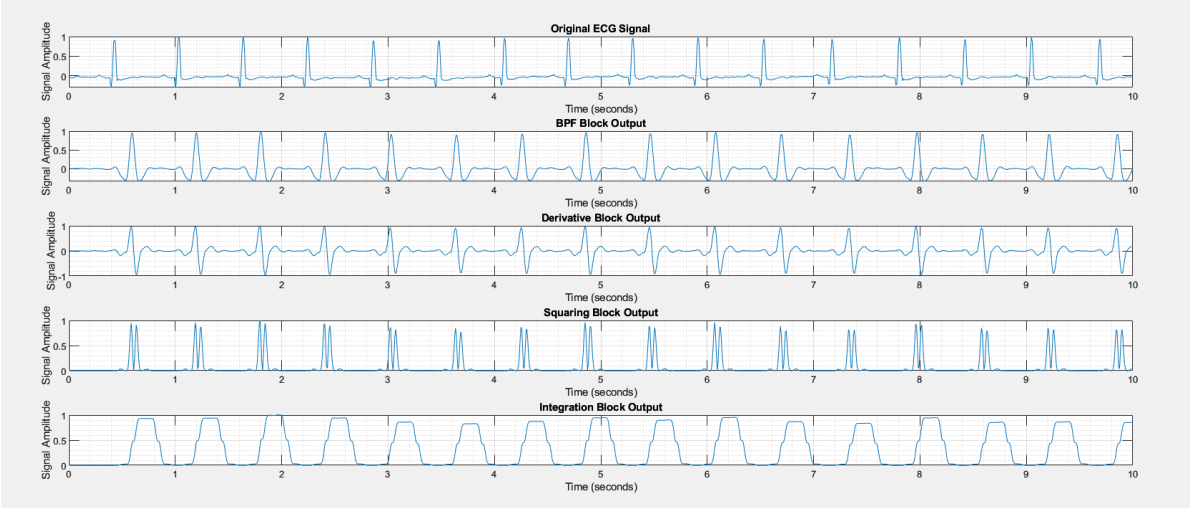
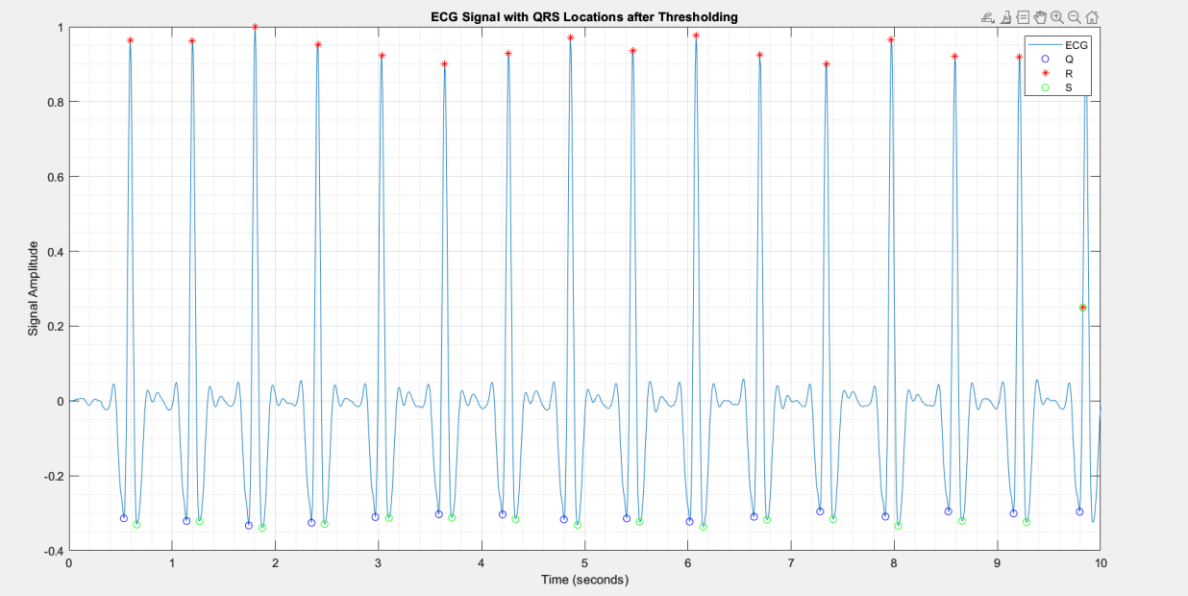


Figure 8 Processing Output




```
Heart Rate for ecg = 96 beats/min

 QRS duration = 0.09668seconds

VF not detected

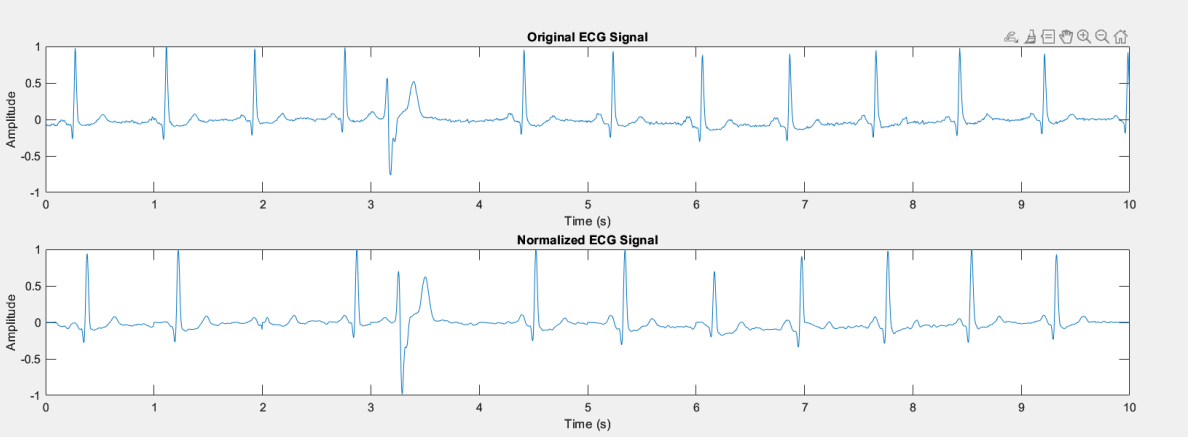
Na =

    6.5755
```

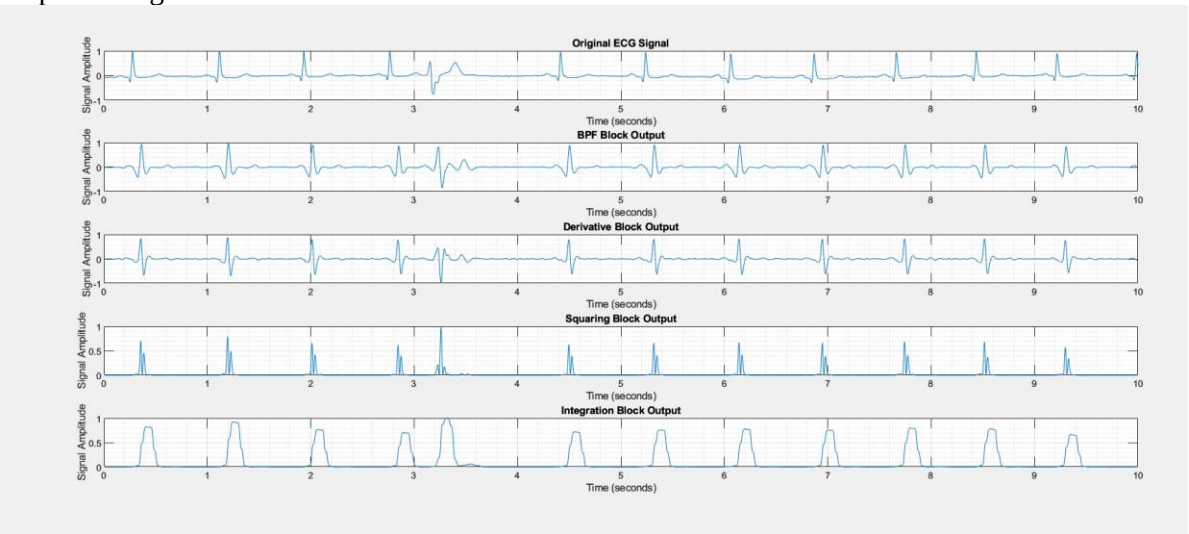
QRS Detection:

Other output:

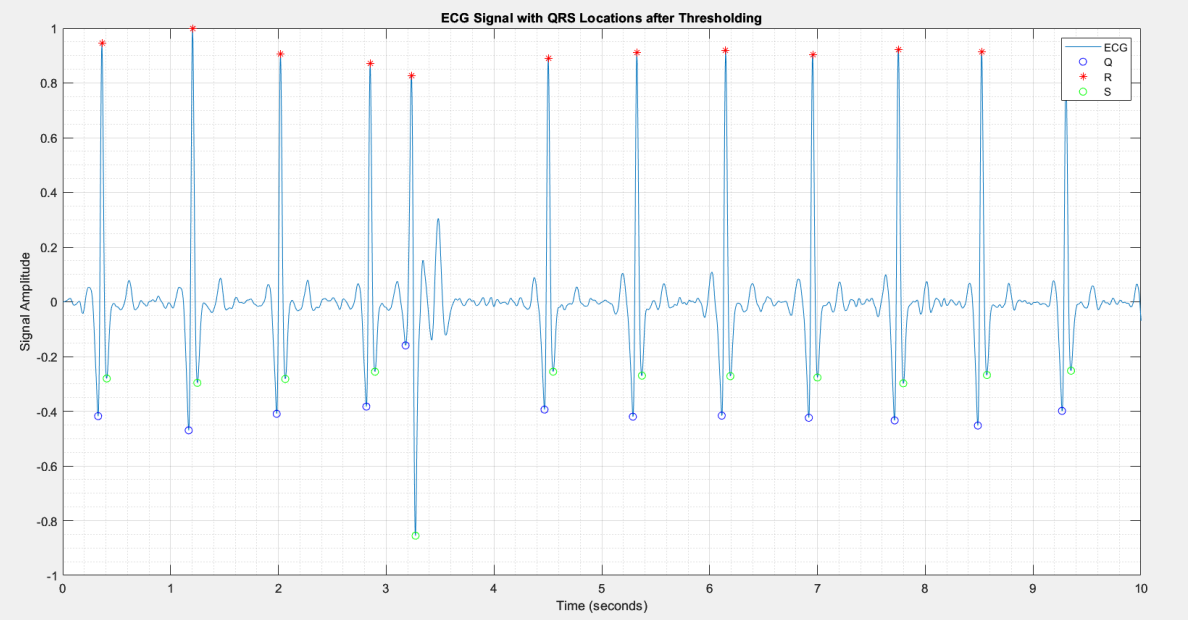
Intput for another Healthy person ECG:



Preprocessing:



QRS detection:



Other Outputs:

```
Heart Rate for ecg = 72 beats/min

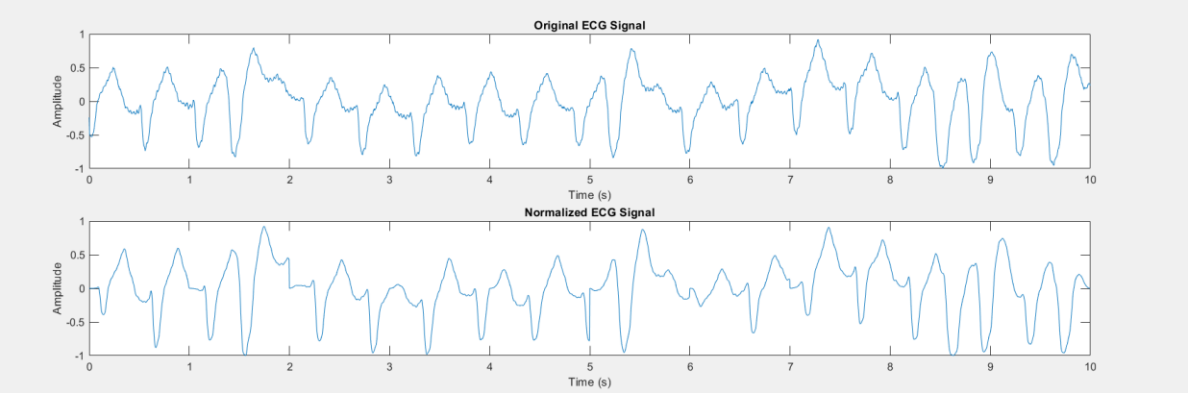
 QRS duration = 0.0445seconds

VF not detected

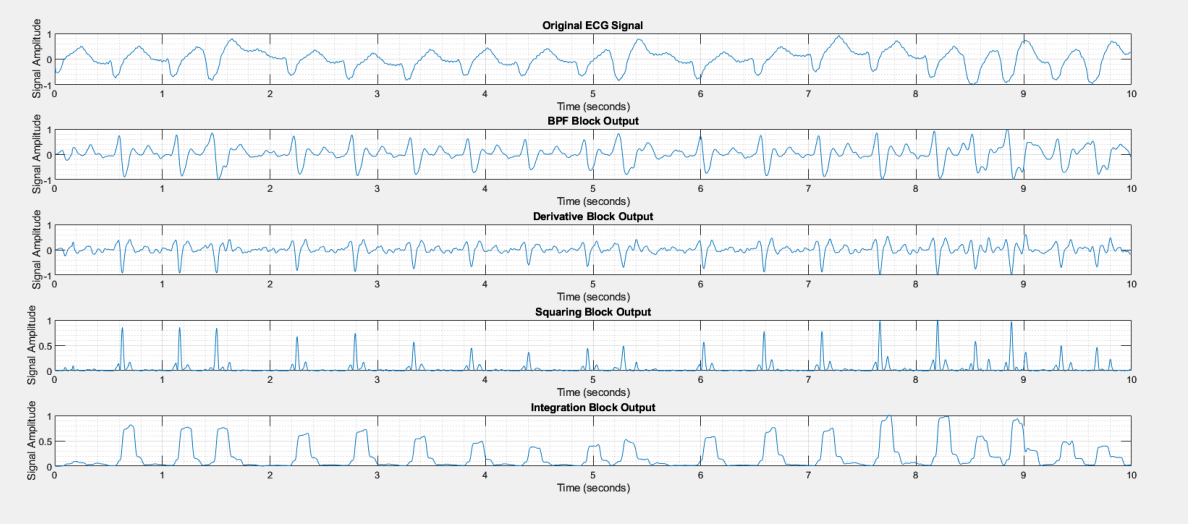
Na =

    5.2500
```

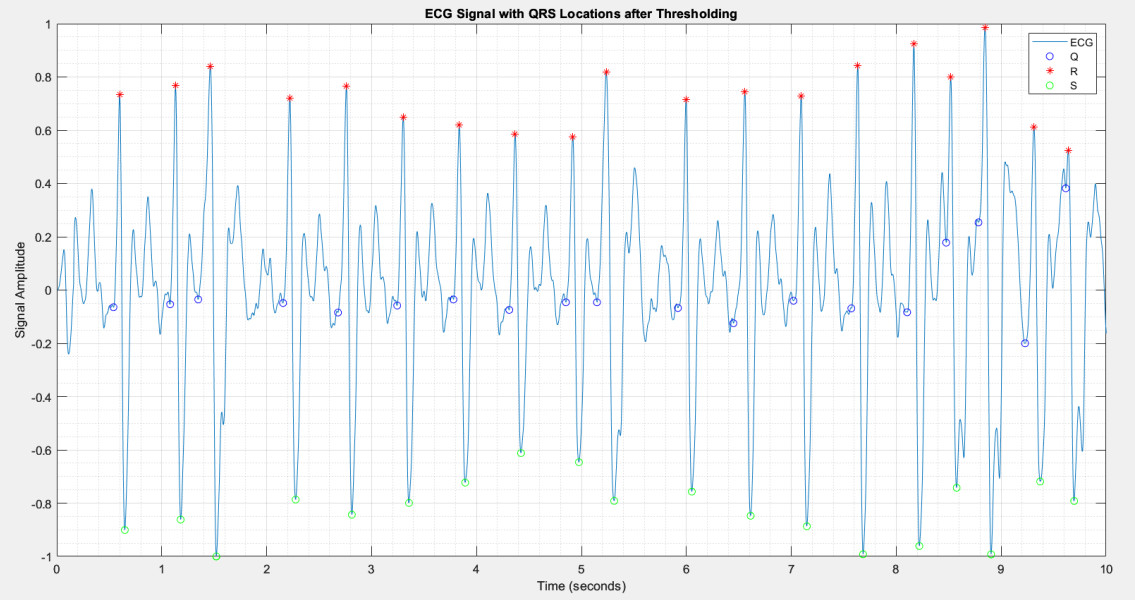
Intput for a Unealthy person ECG:



Preprocessing:



QRS detection:



Other Output:

```
Heart Rate for ecg = 114 beats/min

 QRS duration = 0.027895seconds

VF detected

Na =

   51.3625
```

5 Design Analysis and Evaluation

5.1Limitations of Tools (PO(e))

1. In our algorithm for detecting R-peaks in ECG signals, we employed a Fixed thresholding technique. However, we encountered a challenge where intermediate peaks occasionally surpass the set threshold, leading to the identification of additional R-peaks. This issue cascades into the detection of extra Q and S peaks as well since their determination relies on the accurate identification of R-peaks. The presence of these extra peaks not only complicates the interpretation of the ECG signal but also compromises the precision of subsequent analyses or diagnostic procedures. To mitigate this problem, we need to refine our detection methodology. One approach could involve implementing adaptive thresholding techniques that dynamically adjust the threshold based on the characteristics of the ECG signal. This adaptive approach would enhance the robustness of R-peak detection, reducing the likelihood of false positives and minimizing the occurrence of extra Q and Speaks. Additionally, integrating signal processing techniques such as wavelet transforms or machine learning algorithms could offer more sophisticated means of distinguishing genuine R-peaks from noise or artifact-induced peaks. By enhancing the accuracy of R-peak detection, we can improve the reliability and clinical utility of ECG interpretation and analysis.

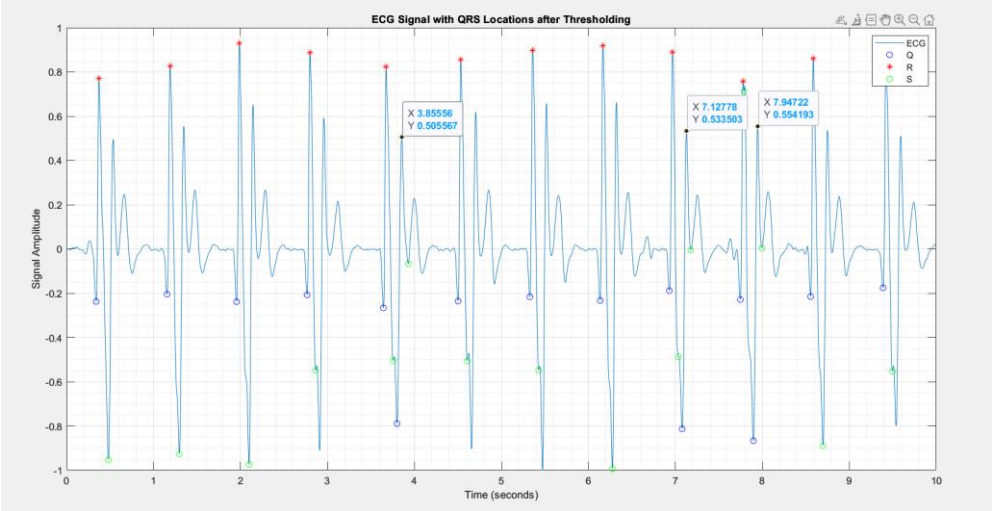
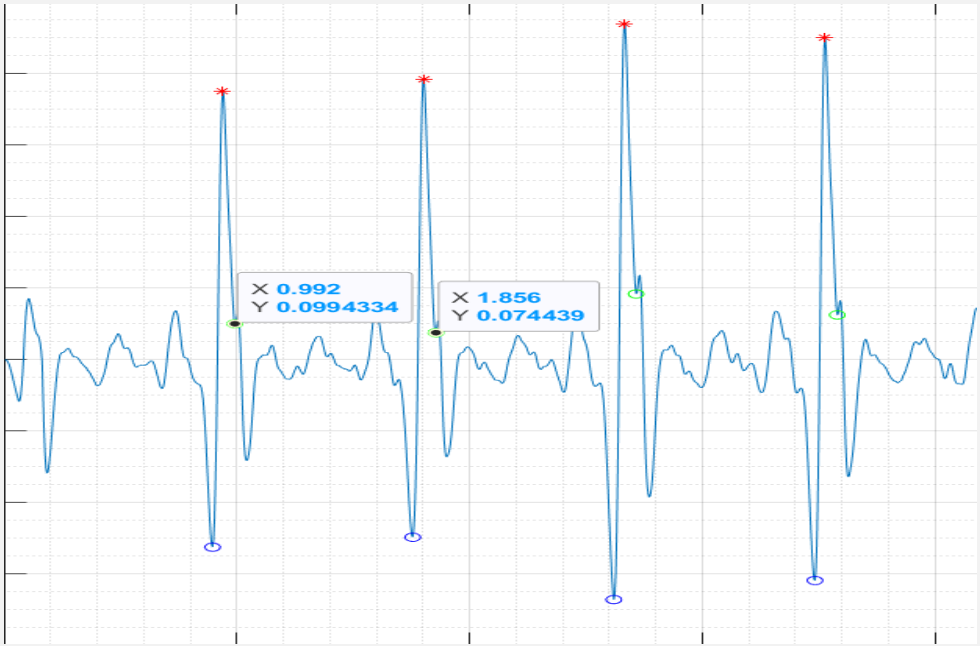


Figure 9 ECG with Q,R,S location

1. In our Q and S peak detection method, we assumed smooth Q-to-R and R-to-S waveforms. However, occasional waveform distortion leads to inaccuracies in identifying precise Q and S peaks. These distortions disrupt the expected smooth transitions, resulting in missed or misaligned peak detections. To address this, we need to refine our algorithm to account for waveform irregularities. Incorporating techniques such as signal filtering or waveform modeling could improve the robustness of Q and S peak detection, ensuring accurate identification even in the presence of distorted waveforms.



6 Reflection on Individual and Teamwork (PO(i))

As mentioned earlier, we have divided it into several functions. Those functions are divided among our team members. The following table shows the individual work:

Name	Work Done
1.Bashir Shahriyer Rafi (2006136)	Loading ECG signal, Noise removal (Band pass filtration), Width of QRS complex Detection of arrhythmia based on QRS complex duration
2. Riajul Karim Chowdhury (2006153)	Highlighting the QRS complex (Derivative filter), Integration, Calculating R-R interval and heart rate
3. Md. Symria Raihan (2006133)	Thresholding (Fixed threshold), Detection of R peak based on threshold, Detection of q & s peak
4. S. M. Razibur Rahman	Moving average system, Detection of VF using time domain approach

7 Future Work (PO(l))

Enhancing the performance of ECG signal processing and ventricular fibrillation (VF) detection is crucial for improving the accuracy and reliability of medical diagnosis and intervention. For implementing these there is no specified algorithm. Day by day easier, faster

and efficient algorithms are being invented. Here are some potential future works for our project:

Refinement of Peak Detection Algorithms
There are many advanced peak detection algorithms such as Wavelet Transform-based methods, Hilbert Transform, or Empirical Mode Decomposition (EMD) [1, 2]. Considering machine learning techniques like support vector machines (SVM), random forests, or neural networks to improve the robustness of peak detection, especially in scenarios with secondary peaks and noisy edges is also a very good idea. [3, 4].

Noise Reduction and Signal Enhancement
Explore signal denoising techniques such as wavelet thresholding, Kalman filtering, or adaptive filtering to mitigate the effects of noise and artifacts on ECG signals [7, 8]. Investigate methods for baseline wander removal and T-wave interference suppression to enhance the clarity of relevant signal components.

Feature Extraction Optimization
Investigate additional features beyond QRS complexes, such as heart rate variability (HRV) parameters, morphology descriptors, and higher-order statistics. Implement feature selection techniques to identify the most discriminative and informative features for VF detection.

Machine Learning Integration
Develop machine learning models, including deep learning architectures like convolutional neural networks (CNNs) or recurrent neural networks (RNNs), to directly classify ECG signals into normal and VF categories [5, 6]. Utilize transfer learning approaches to leverage pre-trained models on large ECG datasets for improved generalization to diverse patient populations and signal variations.

Real-Time Implementation and Deployment
Optimize algorithms for real-time processing and deployment on embedded systems or wearable devices to enable continuous monitoring and early detection of VF episodes. The design of the code and algorithm should be compatible to implement in hardware. If ECG signal detector senses the signal and this signal can be analyzed in real time, then it will be a great project. We can consider hardware acceleration techniques and efficient algorithm implementations to meet the computational constraints of resource-constrained platforms.

Clinical Validation and Evaluation
Conduct extensive validation studies on diverse clinical datasets to assess the performance, sensitivity, and specificity of the proposed algorithms under real-world conditions [9]. Collaborate with healthcare professionals and clinicians to validate the clinical utility and feasibility of the developed VF detection system in clinical practice.

These citations provide foundational references for understanding the theoretical principles and applications of the mentioned algorithms in signal processing and machine learning. Depending on your specific implementation or research context, you may need to refer to additional literature or research papers for detailed methodologies and algorithmic variations.

8 References

[1] Mallat, S. (2009). A Wavelet Tour of Signal Processing: The Sparse Way. Academic Press.
[2] Huang, N. E., Shen, Z., Long, S. R., et al. (1998). The empirical mode decomposition and the Hilbert spectrum for nonlinear and non-stationary time series analysis. Proceedings of the Royal Society of London. Series A: Mathematical, Physical and Engineering Sciences, 454(1971), 903–995.
[3] Cortes, C., & Vapnik, V. (1995). Support-vector networks. Machine Learning, 20(3), 273–297.
[4] Bishop, C. M. (2006). Pattern Recognition and Machine Learning. Springer.
[5] Goodfellow, I., Bengio, Y., & Courville, A. (2016). Deep Learning. MIT Press.
[6] LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. Nature, 521(7553), 436–444.
[7] Donoho, D. L., & Johnstone, I. M. (1994). Ideal spatial adaptation by wavelet shrinkage. Biometrika, 81(3), 425–455.
[8] Mallat, S. (1998). A Wavelet Tour of Signal Processing: The Sparse Way. Academic Press.
[9] (Specific citation depending on the validation study and dataset used).